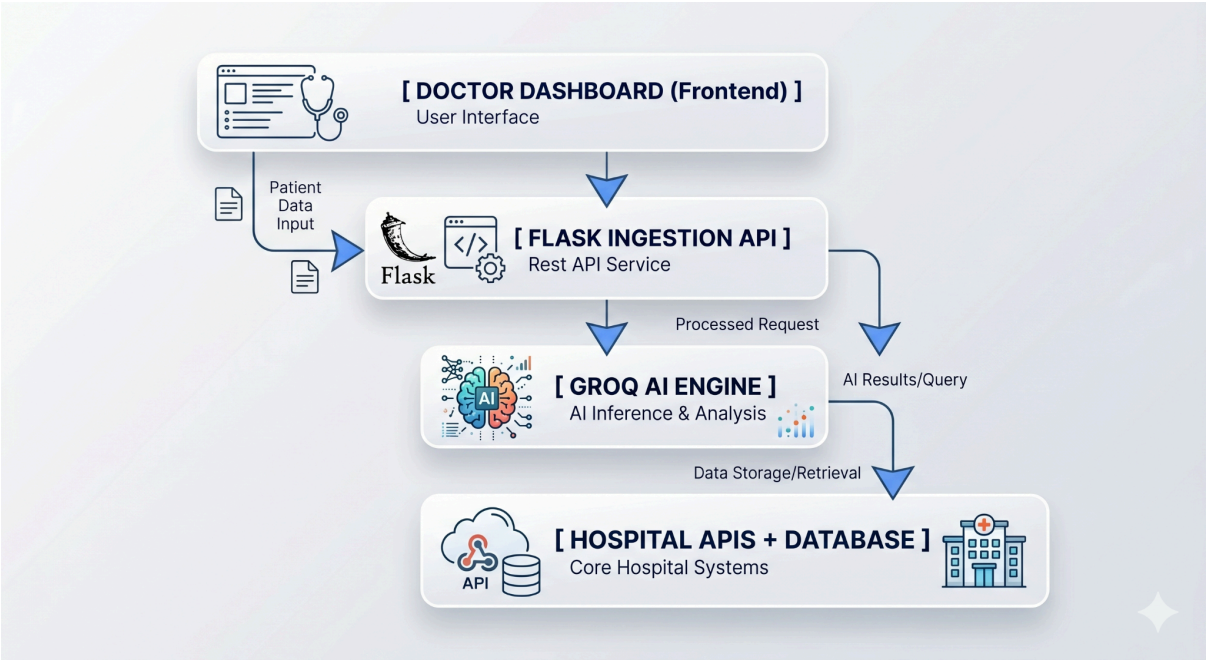
System Architecture Document (SAD)

1.0 System Overview

Ophia follows a **4-layer architecture**:

1. Frontend (Doctor Dashboard)
2. Ingestion Layer (Flask API)
3. AI Engine (GROQ)
4. Data & APIs Layer

Below is a diagram of the system's architecture:



2.0 Component Breakdown

2.1 Frontend (HTML/JS Dashboard)

- Collects user input
- Sends POST request to ingestion API
- Displays results and alerts

2.2 Ingestion Layer (Flask)

- Endpoint: `/api/process-note`
- Handles:
  - form data
  - file uploads

    - text extraction
- Builds JSON contract (Version 2)

2.3 AI Engine (GROQ)

- Performs:
    - entity extraction
    - clinical reasoning
    - workflow decision-making
- Outputs:
    - structured clinical data
    - actions
    - alerts

2.4 Backend APIs

Mock endpoints:

- `/api/reserve-bed`
- `/api/schedule-appointment`
- `/api/log-encounter`

3.0 Data Flow

The system is built to take a doctor's note and turn it into clear, structured information in just a few steps. Whether the note is typed or uploaded, it moves through the system, gets processed by AI, and comes back with useful insights and actions that help support clinical decisions. Below is the overview of the data flow:

1. The user (doctor) submits a clinical note through the dashboard, either by typing text or uploading a file.
2. The ingestion service receives the request and extracts the raw clinical text (from text input, image OCR, or audio transcription).
3. The extracted data is structured into a predefined JSON format and sent to the GROQ-powered processing service.
4. The GROQ service analyzes the note, generates structured clinical data, and determines any required actions or alerts.
5. The processed response is returned to the frontend, where the results, insights, and alerts are displayed to the user.

4.0 JSON Contract

To ensure all components of the system communicate correctly, a fixed JSON structure is used for every request and response. This contract defines exactly how

data is formatted and shared between the frontend, ingestion service, and processing API. Keeping these structures consistent is critical, as any changes can break the integration between components.

Request:

```
{
  "metadata": {
    "doctor_id": "DR-7742",
    "patient_id": "PT-10934",
    "department": "Neurology",
    "timestamp": "...",
    "priority": "high"
  },
  "payload": {
    "input_methods_used": ["text"],
    "raw_text": "..."
  }
}
```

Response:

```
{
  "status": "success",
  "structured_clinical_data": {},
  "actions": [],
  "system_alerts": []
}
```

5.0 Error Handling

To ensure the system remains reliable and safe to use, several error-handling mechanisms are implemented throughout the workflow.

- **Missing Fields** → If required inputs such as patient ID or doctor ID are not provided, the system immediately rejects the request and prompts the user to complete the missing information before proceeding.
- **API Timeout** → If the connection to external services (e.g., AI processing or backend APIs) takes too long or fails, the system returns a fallback message to inform the user that the request could not be completed, preventing the interface from freezing or hanging.

- **AI Ambiguity** → If the AI is unable to confidently interpret the clinical note (e.g., unclear dosage or incomplete information), a **system alert** is generated. This alert is shown to the user, prompting them to clarify or provide additional details before any further actions are taken.